

MUKUND REALITY

TECHNICAL HANDOVER REPORT

Project Parameter	Technical Configuration Data
Project Name	Mukund Reality
Organization Name	Megamind Development Group
Client Organization	Mukund Enterprises Ltd.
Report Compile Date	May 27, 2026
Live Website URL	https://mukundreality.com
Backend Database URL	Shared Firebase/Firestore Instance
Hosting Infrastructure	DigitalOcean VPS (Ubuntu 22.04 LTS)
Total Report Pages	14 Pages
Report Specification Type	Technical Project Handover Report

CONFIDENTIAL

STAGE 3 SECURE HANDOVER DOCUMENT

1. Project Overview

[REPLACE: Provide a high-level overview of the Mukund Reality platform, including its core business goals, target markets, and how it connects property buyers with real estate agents.] The Mukund Reality real-estate platform is a high-performance web solution designed to bridge the gap between luxury home seekers, commercial investors, and property developers. By providing rich interactive maps, immersive image galleries, and detailed property specification listings, the platform functions as the premium digital facade for Mukund Enterprises' diverse real-estate portfolio. The application has been engineered to deliver near-instantaneous page loads, fluid animation timelines, and robust accessibility standards across all device screen resolutions.

[REPLACE: Describe the history of the project development, key milestones achieved, and the overall success of the deployment, including any critical technical decisions made during active development.] Developed as a modern decoupled system, the platform consists of two principal components: a public-facing website optimized for SEO and smooth user experiences, and a secure back-office Content Management System (CMS) Dashboard that empowers administrators to update property catalog details in real-time. Key development milestones included the successful integration of Google Maps clustering, the creation of a zero-overhead database backup script, and the implementation of a secure server-side lead forwarding proxy that communicates directly with the enterprise CRM API.

- Live Website URL: <https://mukundreality.com>
- Backend API URL: Shared Firebase/Firestore Instance
- CMS Dashboard URL: <http://localhost:3001>

1.1 Target Audience

The platform satisfies diverse user segments. The detailed user personas and their primary use cases are summarized in the table below:

Audience Segment	Primary Use Case
Prospective Buyers / Students	Researching real estate listings, project amenities, local proximity maps, and pricing structures.
Parents / Investors	Evaluating financial details, security infrastructure, construction updates, and site plans.
Current Tenants / Students	Accessing announcements, maintenance tickets, and service updates from the CMS.
Alumni / Communities	Submitting reviews, checking development portfolios, and browsing organizational milestones.
Corporate Recruiters / Partners	Reviewing corporate team statistics, careers listings, and contacting administration.
Evaluating / Auditing Bodies	Inspecting technical compliance documents, registration certificates, and developer history.

2. Technology Stack

To achieve modern user experience expectations, rapid responsiveness, and secure admin control, a unified technology stack has been chosen for both the public-facing Website and the CMS Dashboard.

2.1 Frontend Technologies

The frontend incorporates modular packages, advanced scrolling physics, and real-time database listeners compiled under Next.js 16:

Technology	Version	Purpose / Usage
Next.js	16.1.6	Core React framework utilizing the App Router for static pre-rendering and dynamic layouts.
React	19.2.3	Component-based declarative UI generation engine.
Firebase Client SDK	12.9.0	Client-side authentication, local storage uploads, and real-time Firestore listeners.
Swiper	^12.1.0	High-performance responsive touch slide galleries for properties.
Framer Motion	^12.34.0	Fluid page transitions, component entry animations, and hover state triggers.
GSAP	^3.14.2	Advanced timeline-based scroll parallax effects and morphing elements.
Lenis	^1.3.17	High-performance smooth scroll library ensuring luxury navigation feel.
Lucide React	^0.563.0	Optimized vector icons exported as treeshaken React components.
TailwindCSS	^4.0.0	Utility-first modern CSS compiler coupled with PostCSS processes.
TypeScript	^5.0.0	Type safety layer facilitating clean refactoring and solid developer contracts.
React Email Editor	^1.7.11	WYSIWYG layout builder utilized in the CMS to construct customer newsletters.
React Quill New	^3.8.3	Rich Text editor enabling HTML blogging and company announcements in the CMS.

2.2 Backend & API Architecture

The backend leverages Serverless Cloud Functions, direct Firebase Admin sockets, and external proxy APIs. This allows safe storage reads and writes while preventing key exposure on public pages:

Component	Technology	Details / Connection Pattern
Backend API Server	Next.js Server Actions	Executes secure data mutations and server-side mailers directly on the server layer.
Database Engine	Google Cloud Firestore	NoSQL document database providing real-time data sync across both Website and CMS.
HTTP Client	Axios & Fetch API	Issues REST requests to secure CRM endpoints and external email delivery providers.
API Proxy Routes	Next.js Route Handlers	Internal <code>`/api`</code> endpoints proxying traffic securely to avoid CORS errors.

Component	Technology	Details / Connection Pattern
Admin SDK Sockets	firebase-admin ^13.6.1	Privileged CRUD operations, backups, and user credentials editing inside the CMS.
Static Data Files	Pre-compiled JSON	Fast local asset manifests for constant components (e.g. project configurations).

2.3 Infrastructure & DevOps

A resilient, production-ready virtualized infrastructure has been provisioned to serve both applications safely:

Infrastructure Tier	Technology	Role & Configuration
Cloud Host VM	DigitalOcean Droplet	A virtual private server sandbox executing application node runtimes.
Reverse Proxy Server	Nginx Engine	Handles incoming port 80/443 traffic, path-based proxy routing, and TLS termination.
Process Manager	PM2 Daemon	Monitors node instances, schedules restarts, and handles daily logging files.
SSL Certificates	Let's Encrypt CA	Validates server ownership and generates 256-bit ECDSA certificates for secure HTTPS.

3. System Architecture

3.1 Architecture Pattern

[REPLACE: The Mukund Reality application follows a modern decoupled architecture. The frontend is built on Next.js 16 (using the App Router) which serves static pages, pre-rendered components, and dynamic client-side pages. The backend utilizes Next.js Server Actions and Firebase Admin SDK, interfacing with a Firestore NoSQL database for real-time updates and transactional data processing. State management is kept lean using React Context and standard state hooks, ensuring optimal load performance and rendering times.] The system architecture is built on Next.js 16 utilizing the modern React App Router paradigm to implement a hybrid rendering strategy. Data synchronization is managed via Google Cloud Firestore, allowing the CMS dashboard to write real-time updates that are immediately visible on the public website. Secure transactions and file storage operations are encapsulated inside server-only Next.js API Routes and Server Actions utilizing the `firebase-admin`

SDK, ensuring database credentials and API tokens are never exposed to client browsers.

3.2 Rendering Strategy

Next.js App Router utilizes individual page configurations to achieve the optimal speed and SEO indexation for different routes:

Strategy	How It Works	Applied To
SSG (Static Site)	HTML is generated once at build-time. Zero database hits at runtime.	Invariant pages: /about, /contact, /privacy-policy
ISR (Incremental Static)	Statically built but re-compiles silently in background upon traffic after expiry.	Dynamic content: /insights, /careers, /company-announcements
SSG + CSR (Hybrid)	Delivers standard HTML structure immediately; hydration fetches user data dynamically.	Real-time components: /projects listing page and /project-enquire
SSG via generateStaticParams	Builds dynamic routes dynamically using predefined listing slugs.	Individual property landing pages (e.g. /evanna-homes, /rudraksh)

3.3 Complete Page Route Table

The following table maps the entire site directory routes for both the Public Website and CMS Administrative Dashboard:

Route Pathway	Rendering Mode	Primary Data Source
/ (Root Home)	SSG + ISR (60s)	Firestore (`projects` collection)
/about	SSG	Local static JSON Config
/projects	SSG + CSR	Firestore client reads (dynamic filters)
/contact	SSG	Form submissions written to CRM API & Firestore
/careers	SSG + ISR (1hr)	Firestore (`careers` collection)
/insights	SSG + ISR (1hr)	Firestore (`insights` collection)
/company-announcements	SSG + ISR (5m)	Firestore (`announcements` collection)
/project-enquire	SSG + CSR	Client input (writes to Intelitech CRM API)
/api/crm	POST Route	Proxies lead data to mgmcrm.intelitechsolutions.in

Route Pathway	Rendering Mode	Primary Data Source
/admin (Dashboard)	CSR (Protected)	Firestore stats summaries & enquiries counts
/admin/login	CSR	Firebase Auth client verification
/admin/projects	CSR (Protected)	Firestore CRUD operations (`projects` collection)
/admin/enquiries	CSR (Protected)	Firestore streams (`projectEnquiries` & `generalEnquiries`)
/admin/careers	CSR (Protected)	Firestore CRUD (`careers` & job applications)
/admin/blogs	CSR (Protected)	Firestore CRUD (`blogs` collection)
/admin/announcements	CSR (Protected)	Firestore CRUD (`announcements` collection)

3.4 Animation Architecture

Visual aesthetic interactions are driven by combining declarative timelines and scroll physics engines:

Animation Library	Primary Architectural Role	Applied Components
Framer Motion	Handles localized reactive UI animations, page transitions, and staggered list entries.	Mobile navigation dropdown, form inputs, button interactions, fade-in-up cards.
GSAP & ScrollTrigger	Orchestrates highly complex timelines and scroll-driven displacement patterns.	Hero sections parallax, smooth property details scroll timelines, interactive maps pinning.

3.5 Request-to-Render Data Flow

The process diagram below depicts the execution lifecycle when a client requests a listing page and submits an enquiry:

Step	Execution Layer	Action & Lifecycle Event
1	Client Browser	User enters URL path `/evanna-homes` into the browser and presses Enter.
2	Nginx Proxy	Proxy intercepts request, verifies SSL, and queries Next.js cached static directories.
3	Next.js Server	Statically generated HTML shell and precompiled Swiper scripts are delivered in milliseconds.
4	React Hydration	React hydrates DOM, executing client-side GSAP timelines and initializing smooth scrolling.

Step	Execution Layer	Action & Lifecycle Event
5	Firestore API	Client-side Firestore sockets fetch real-time gallery modifications that occurred post-build.
6	Lead Capture	User submits an enquiry; Next.js server actions save to Firestore and proxy to Intelitech CRM.
7	Admin Alerts	CMS Dashboard captures Firestore update event, sending push notification to admins.

4. Hosting & Infrastructure

4.1 Server Instance Details

The hardware specification and OS configuration for the centralized staging/production VM are detailed below:

System Parameter	Configured Production Value
Cloud Hosting Provider	DigitalOcean (VPS Droplets)
Service Tier	General Purpose Droplet
Server Region	Bangalore, India (BLR1)
Operating System	Ubuntu Linux 22.04 LTS (x86_64)
Public IP Address	159.89.172.48 (DNS binds to mukundreality.com)
Total Storage Allocation	50 GB NVMe Solid State Drive
System Memory RAM	2 GB DDR4 ECC RAM
Swap File Space	2 GB Configured for safe builds (prevents out-of-memory errors)
Node.js Engine Version	v20.12.2 (LTS Iron)
npm Package Version	v10.5.0

4.2 Application Deployment Configuration

Process configurations managed by PM2 run continuously as service daemons inside the droplet:

Deployment Property	Configured Service Details
Application Namespace	Mukund Reality (Website & CMS Dashboard)
PM2 Process Names	`mukund-website` (Port 3000) & `mukund-cms` (Port 3001)
Internal Running Ports	Port 3000 (Website), Port 3001 (CMS Dashboard)
Public Access Domains	website.mukundreality.com cms.mukundreality.com
Static Build Command	`npm run build` (Pre-compiles Next.js dynamic chunks)

Deployment Property	Configured Service Details
PM2 Startup Invocation	<code>`pm2 start npm --name "mukund-website" -- run start`</code>

4.3 Reverse Proxy Configuration (Nginx)

Nginx serves as the gateway proxy, routing public requests to local server ports:

Nginx Configuration Property	Value / Path Location
Active Configuration File	/etc/nginx/sites-available/mukund-reality
Domain Server Names	website.mukundreality.com (Website), cms.mukundreality.com (CMS)
Proxy Pass Targets	http://localhost:3000 (Website) http://localhost:3001 (CMS)
Security Protocols Enabled	HTTP/2, TLSv1.3, Strict-Transport-Security (HSTS)
HTTP Redirection Rules	Forced HTTP-to-HTTPS redirect (Rewrites Port 80 requests to Port 443)

4.4 SSL / TLS Configuration

Automated security profiles secure communications to protect client leads:

Security Property	Active TLS Configuration Details
Certificate Authority (CA)	Let's Encrypt Authority
Automated Provisioner	Certbot ACME Client Engine
Certificate Type	ECDSA 256-bit Key Length (High Performance SSL)
TLS Protocols Enforced	TLSv1.2 and TLSv1.3 (Modern Ciphers Only)
Validity Duration	90 Days standard period
Renewal Mechanism	Automated systemd timer executing twice daily check (<code>`certbot renew`</code>)
Secured Host Domain	website.mukundreality.com, cms.mukundreality.com

4.5 Process Management & Daemon Resiliency

PM2 guarantees application availability. The configured features are described below:

Daemon Feature	Implementation & Operational Details
Auto-Restart Safeguard	Restarts processes instantly if memory usage exceeds 1.2GB or an uncaught exception is thrown.
Startup Persistence	Registered with <code>`systemd`</code> daemon. Restores website and CMS on server boot automatically.

Daemon Feature	Implementation & Operational Details
Automated Log Rotation	Integrated with <code>`pm2-logrotate`</code> to prevent disk saturation by rotating logs daily at 10MB.
Operational Monitoring	CLI dashboard available via <code>`pm2 monit`</code> providing CPU, RAM, and tick rates statistics.
Zero-Downtime Releases	Deploys code updates using <code>`pm2 reload`</code> which performs graceful staggered connection handovers.

5. Features

The platform packages enterprise-level real estate solutions. The key operational features covering both Website and CMS projects are listed in the table below:

Feature Title	Functional System Description
Interactive Proximity Maps	Integrates Google Maps API displaying dynamic customized listing pinpoints, clustered geographically based on coordinate fields.
Multi-Criteria Filter Engine	Client-side filtration querying Firestore records instantly by price, location, categories (commercial/residential), and project stage.
Swiper Property Media Galleries	Rich carousel galleries displaying construction renders, floor layouts, and completed project images with lazy loading.
Real-time Enquiry Forms	Lead fields capture forms saving immediately to Firestore <code>`projectEnquiries`</code> and syncing to CRM API endpoints.
Job Listings & Resumes	Renders career vacancies; allows applicants to upload resumes, storing PDFs securely in Firebase Storage.
Urgent Banner Banners	Dynamic site-wide emergency/announcement banner managed directly from the CMS announcements panel.
Real Estate News Blogs	SEO-optimized publishing platform featuring categories, author tags, and formatted content fields for real-estate insights.
Dashboard Data Analytics	CMS homepage aggregates listing totals, unread enquiry numbers, and career files count via custom stats APIs.
WYSIWYG HTML Editing	CMS blog builder integrates React Quill to format paragraphs, insert links, and manage headings smoothly.
Real-time CRUD Control	Administrators can add, update, or archive projects, agents, and careers with immediate public-facing visibility.
Automated Firestore Backup	CLI script <code>`npm run db:backup`</code> serializes active Firestore collections to <code>database_backup.json</code> locally.
Instant Database Seeding	<code>`npm run db:seed`</code> script restores complete project and team data sets from backup files onto blank Firestore instances.

6. API Integration

6.1 Data Fetching Endpoints

The platform interacts with the shared database using structured API pathways, balancing static caching and real-time synchronization:

API Pathway	Caching & Revalidation	Functional System Purpose
/api/announcements	ISR (Revalidate: 300s)	Fetches active site-wide announcement banners.
/api/blogs	ISR (Revalidate: 3600s)	Retrieves catalog of published blog and news posts.
/api/careers	ISR (Revalidate: 3600s)	Fetches active job openings and descriptions.
/api/enquiries	Dynamic (Cache: None)	CMS Admin dashboard endpoint to review customer submissions.
/api/insights	ISR (Revalidate: 3600s)	Retrieves property advice, news, and market insights.
/api/stats	Dynamic (Cache: None)	CMS Analytics panel summarizing collection metrics.
/api/team	SSG (Build-time Cache)	Fetches list of core management partners and staff.
firestore/projects	Real-time SDK Listener	Direct client connection displaying latest listings on `/projects` page.
firestore/projectEnquiries	Direct Client Write	Permitted collection write to insert property leads from Website forms.
/api/crm (Internal Proxy)	Dynamic (POST Only)	Safe serverless gateway forwarding user leads to the corporate CRM API.

6.2 Form Submission Endpoints

User forms submitted on the Website are securely handled, saving the record locally in Firestore and forwarding immediately to the CRM:

Submission Route	Payload Signature	Operational Business Purpose
/api/crm	`name`, `email`, `phone`, `project`, `interestedIn`	Proxies lead data to Intelitech solutions CRM (mgmcrm.intelitechsolutions.in) for immediate automated sales team distribution.
/api/enquiries (Direct WRITE)	`name`, `email`, `phone`, `message`, `type`, `timestamp`	Registers general site queries or contact requests directly into Firestore collections for administrative review.
/api/careers (Direct WRITE)	`name`, `email`, `position`, `phone`, `cvUrl`, `coverLetter`	Saves job candidate application files, uploading PDF files securely to Firebase Storage buckets.

7. Performance & Optimization

7.1 Image Optimization Protocols

Image sizes are optimized using Next.js asset compilation to minimize mobile loading delays:

Optimization Feature	Active Technical Implementation Details
Next.js Image Component	Enforces dynamic source scaling and converts raw PNG/JPG assets to modern AVIF/WebP formats.
Viewport Lazy Loading	Images are flagged with priority parameters. Static assets load only as they enter the screen viewport.
Lossless Compression	Restricts Firebase Storage uploaded listing images to a optimized 75% quality compression threshold.
CDN Edge Caching	Enforces cache-control metadata rules on Firebase storage headers to keep images cached at Cloudflare/Google CDN edges.
Responsive Layout Srcsets	Generates source sets targeting specific media query breakpoints, serving compact dimensions to mobile viewports.

7.2 Caching Strategy tiers

Caching layers are implemented to isolate database nodes and speed up recurring request speeds:

Cache Layer	Implementation Strategy	Project Performance Benefit
ISR Server Cache	Statically generated pages (Next.js server cache directory) utilizing custom revalidate periods.	Drastically cuts down Firestore API lookup fees while returning static compilation speeds.
SSG Build Cache	Statically compiled HTML/CSS assets built once during build sequence.	Returns instantaneous response times, reducing host VPS load during traffic surges.
Browser Local Cache	Configures `Cache-Control: max-age=31536000` headers on constant fonts, assets, and icons.	Bypasses network connection completely for repeat visitors, making transitions feel instant.
Image Optimizer Cache	Preserves compressed layouts inside local server `.next/cache/images` directory.	Eliminates duplicate CPU load cycles associated with resizing large listing graphics.

8. SEO & Discoverability

Comprehensive search engine optimization rules are built directly into standard page layouts, ensuring discoverability:

SEO Optimization Element	Technical Implementation Pattern
Semantic Markup Standards	Uses HTML5 tags (header, main, section, article, footer) helping crawl robots map content hierarchies easily.

SEO Optimization Element	Technical Implementation Pattern
Strict Heading Chronology	Validates that every single page contains exactly one high-level `h1` element, followed by nested `h2`, `h3` tags.
Next.js Metadata Objects	Dynamic metadata templates defined per page route to inject contextual titles, meta descriptions, and keyword structures.
Open Graph Protocol	Configured og:title, og:description, and og:image tags rendering rich link previews on social platforms.
XML Sitemap Generator	Auto-generated at `/sitemap.xml` listing all property project slug URLs to guarantee rapid Google indexation.
Robots Exclusion Rules	Configures /robots.txt directing spiders to crawl landing routes while strictly blocking administrative `/admin` subdirectories.
Mandatory Alt Attributes	Asserts image descriptors on listing graphics and Swiper sliders to score 100% on web accessibility tools.
Canonical Page References	Appends dynamic canonical URLs, eliminating indexing penalties associated with duplicate dynamic filters lists.

9. Security

Robust server-side isolation, access tokens encryption, and client permissions locks protect system integrity:

Security Control Layer	Technical Implementation Pattern
Firestore Security Rules	Locks database. Blocks all public write attempts to critical collections (`projects`, `announcements`, `careers`) while allowing user lead writes.
Firebase Client Authentication	Administrative accounts are protected inside Firebase Auth. Requires verified session tokens before displaying CMS routes.
Environment Token Isolation	Delicate private keys (including firebase-admin keys) are isolated inside local server `.env` and `.env.local` files.
Internal Server API Proxying	CRM credentials and endpoints are hidden. Client calls `/api/crm` and the server forwards securely without key leakage.
Content Security Policy (CSP)	Configures strict header declarations inside next.config.ts limiting script injections and avoiding XSS vectors.
Force HTTPS Rules	Nginx server redirects all standard port 80 traffic to port 443, securing credentials using TLSv1.3 and HSTS headers.

10. Frontend Architecture

10.1 Component Structural Organization

The directories are structured logically to guarantee scalability and maximize code reusability:

Directory Pathway	Architectural Role & Purpose
-------------------	------------------------------

Directory Pathway	Architectural Role & Purpose
src/components/ui	Contains atomic, design-system-specific visual components (buttons, badges, inputs, text grids) styled with standard utility CSS.
src/components/common	Encapsulates global, layout-specific visual elements shared across routes (Header navbar, footer, error modals).
src/components/features	Groups highly specialized interactive blocks (ProjectSwiper galleries, GoogleMapView instances, EnquiryModals).
src/lib	Houses custom framework configuration instances (Firebase initialization, Lenis scrolling instances, GSAP timeline defaults).
src/utils	Stores pure-logic helper functions (currency formatters, calendar date parsers, string truncators).

10.2 State Management Pattern

[REPLACE: State management within the Mukund Reality frontend relies on a combination of React Context for global app states (such as authentication state, listing filters, and theme preferences) and React's built-in useState/useReducer for localized component state. This keeping-it-simple approach ensures that bundle sizes remain lightweight and standard Next.js page transitions remain lightning fast.] The application implements a highly efficient, lightweight state management pattern. Global states—such as client-side user sessions, active administrative configurations, and listing filters—are distributed using lightweight React Context Providers. Local user-interface behaviors, scroll positions, and form inputs are controlled locally within isolated components using `useState` and `useReducer` hooks, maximizing rendering performance and eliminating heavy state-library bundle overhead.

State Profile	Management Approach	Operational Details
Admin User Session	React Context Provider + Auth SDK	Streams active session states across CMS routes. Triggers auto-redirect to `/admin/login` on token expiry.
Listing Filter Properties	React Context Provider	Buffers selected property categories, budget limits, and regions, updating listing grids in real-time.
Client Form Input Buffer	Component Local State (`useState`)	Keeps temporary track of key strokes and form fields locally before dispatching package payload to the API.
Swiper Active Slide Index	Component Local State (`useState`)	Registers current property image slide indices to sync visual captions and navigation nodes.

11. Environment Configuration

Environment variables are safely compartmentalized. Secret variables are mapped below:

Environment Key Name	Configured Variable Value	Functional System Purpose
----------------------	---------------------------	---------------------------

Environment Key Name	Configured Variable Value	Functional System Purpose
NEXT_PUBLIC_FIREBASE_API_KEY	AlzaSyBAWxEWD4FZhaaAMrX0bOhjDi... [MASKED]	Authorizes client-side connections to Firebase, enabling real-time listings synchronization.
FIREBASE_PRIVATE_KEY	-----BEGIN PRIVATE KEY-----\nMII... [MASKED]	Grants elevated database CRUD permission credentials to administrative scripts in the CMS Dashboard.

REPORT PREPARED BY

Megamind Development Group

Initiated on: 27, 2026

This document is classified as CONFIDENTIAL and contains proprietary system settings.

12. Live Domain Summary

The operational host domains for the complete real estate platform are listed below:

Public Bind Domain	Active Running Application	Local Port	SSL Certificate Status
mukundreality.com	Mukund Reality Public Website	Port 3000	Secured (Let's Encrypt SSL, Auto HTTP Redirect)
cms.mukundreality.com	Mukund Reality CMS Dashboard	Port 3001	Secured (Let's Encrypt SSL, Auto HTTP Redirect)

